Improving Long-Term Oil Price Forecasts

Final Project Report

INDENG 290 Energy Project
May 11, 2026

Team Members
Zhangrui Ji
Jinrong Yan
Dahai Huang
Jingyue Shao

Repository: https://github.com/jingrong129/INDENG-290

This report uses the newly uploaded GitHub repository as the live appendix for source code, data, figures, processed outputs, and reproducibility materials.

Executive Summary

This project evaluates whether long-horizon oil price forecasts are useful for upstream oil and gas investment planning. The stakeholder is an upstream planning team that must decide how to screen projects, how conservatively to price future cash flows, and whether public institutional outlooks can be used directly in capital allocation. We focus on the Energy Information Administration Annual Energy Outlook Brent price vintages because they provide the cleanest public long-horizon archive for a reproducible vintage evaluation.

The core analytical question is whether EIA long-term Brent forecasts add information relative to a naive no-change benchmark at horizons relevant to project screening and development timing. We evaluate three-year-ahead and five-year-ahead annual average Brent prices because those horizons align better with upstream appraisal, development sequencing, and final investment decisions than short-term spot forecasts do.

The project constructs a reproducible panel of EIA forecast vintages from 2013 through 2026, excluding 2024 because no AEO 2024 vintage was released. EIA forecasts are paired with realized annual Brent spot prices from FRED and converted into real 2025 dollars using CPI. Forecast performance is measured with mean error, mean percentage error, mean absolute error, root mean squared error, and mean absolute percentage error. The benchmark is a martingale-style no-change forecast equal to the most recently completed annual Brent price available at the forecast release date.

The main result is nuanced. EIA beats the martingale benchmark on RMSE at both horizons, with 3-year RMSE of 34.97 compared with 42.99 for the benchmark and 5-year RMSE of 34.32 compared with 47.34. At the same time, EIA forecasts are systematically optimistic: mean error is +20.44 dollars per barrel at three years and +26.64 at five years. A recursive historical bias correction improves accuracy, especially at the five-year horizon where RMSE falls to 25.36. Simple rolling autoregressive and futures-augmented models do not beat EIA in this implementation.

The recommendation is to use EIA as an informational baseline but not as a literal planning deck. The EIA series appears to contain useful information, so discarding it would be wasteful. However, the realized sample shows enough optimism that using the published forecast mechanically could overstate project value. A practical planning process should start from EIA, apply explicit conservative adjustment or bias correction, and document the adjustment so decision makers understand when the internal planning deck differs from the public institutional forecast.

Scope and Stakeholder Context

The primary stakeholder is an upstream oil and gas company, especially strategy, planning, corporate finance, and new ventures teams. These teams rely on long-term commodity price assumptions when they screen exploration opportunities, evaluate project economics, compare capital allocation alternatives, and communicate assumptions to senior management. The report is also relevant to lenders, reserve-based financiers, private equity sponsors, and analysts who need to understand how much confidence to place in public long-term oil price outlooks.

The stakeholder’s decisions are not abstract statistical choices. They include whether a project advances past screening, whether a development case remains attractive under a standardized price deck, how much downside protection should be required before sanctioning capital, and whether an institutional public forecast can be adopted directly. A

persistent positive bias in long-horizon oil forecasts can translate into overstated revenue, inflated NPV, optimistic IRR, and poor ranking of marginal projects. Conversely, if the institutional forecast contains information beyond a naive benchmark, discarding it can leave useful signal on the table.

This project therefore treats forecast evaluation as a decision-support exercise. The goal is not simply to identify a statistically elegant model. The goal is to determine whether a planning team should trust EIA long-term Brent forecasts, whether it should adjust them, and what type of adjustment is most defensible given public data and the limited realized sample.

Research Questions

The report addresses five linked questions. First, do EIA long-horizon Brent forecasts contain useful information relative to a martingale no-change benchmark? Second, are the forecasts systematically biased, and if so, how large is the bias in both dollars per barrel and percentage terms? Third, can simple rolling econometric models outperform the institutional forecast? Fourth, does a recursive historical bias correction improve practical forecast performance without look-ahead bias? Fifth, does converting nominal and vintage-dollar quantities into real 2025 dollars create interpretive issues for the Brent analysis?

These questions are deliberately practical. The benchmark is transparent enough that a planning team could implement it without specialized modeling. The bias correction is simple enough to govern and explain. The rolling models test whether public price history and a front-month futures proxy can improve on institutional forecasts. The CPI diagnostic checks whether the real-dollar standardization supports comparability without becoming the driver of the substantive result.

Data Sources and Coverage

The main forecast data source is EIA AEO Table 12, which contains petroleum price forecasts. The pipeline stores local raw workbooks for vintages 2013 through 2023, 2025, and 2026. AEO 2024 is excluded because that release does not exist in the archive. Each workbook is parsed for the Brent Spot row, the target-year columns, and the table's constant-dollar base year. The code then maps each vintage into a common real 2025 dollar unit.

Realized annual Brent prices come from FRED series DCOILBRENTEU. Daily observations are averaged by calendar year. CPI comes from FRED series CPIAUCSL and is averaged to annual CPI for real-dollar conversion. The market-information extension uses Yahoo Finance Brent front-month futures ticker BZ=F. The World Bank commodity price forecast archive is included as supplementary material because it provides another public oil-price forecast history, although its object is crude oil average rather than exact Brent.

The realized evaluation sample contains 20 observations across the 3-year and 5-year horizons. This sample is necessarily small because long-horizon forecast evaluation requires target years to have already occurred. The small sample is a limitation, but it is also typical of vintage forecast evaluation in long-horizon energy planning. The report therefore emphasizes transparent metrics, robustness checks, and decision interpretation rather than overfitting a complex model.

Data Processing and Unit Standardization

Pipeline Flow Chart

Figure 1 summarizes the reproducible workflow used in the GitHub repository: raw forecast and market data are standardized into real 2025 dollars, matched into a 3-year and 5-year evaluation panel, and then passed into benchmark, model-extension, and diagnostic analyses before final tables and figures are exported.

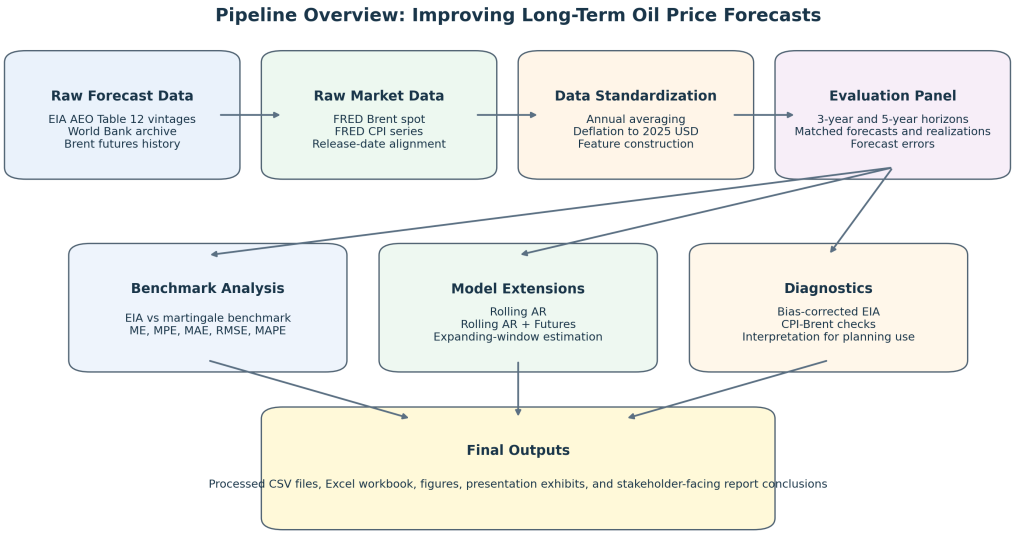

Figure 1. Project pipeline flow chart from raw data through final outputs.

The first processing step is to turn heterogeneous vintage files into a structured panel. For each AEO vintage, the parser locates the year header row, identifies the Brent Spot row, records the table's real-dollar base year, and stores one record for each vintage-target pair. The code preserves release dates so that benchmark and model features are aligned with information that would have been available at the time of forecast publication.

The second processing step converts all price series into real 2025 dollars. Annual CPI is computed from monthly CPI, and the conversion factor for each year equals the 2025 CPI average divided by that year's CPI average. EIA forecasts reported in their table-dollar base year are multiplied by the corresponding base-year factor. Realized annual Brent prices are multiplied by the target-year factor. This standardization makes older and newer forecasts comparable in a common purchasing-power unit.

The third processing step builds the benchmark and evaluation panel. For each vintage-target pair, the forecast horizon is target year minus vintage year. The report keeps 3-year and 5-year observations with realized actuals. The martingale benchmark uses the latest completed annual Brent price available at the forecast origin, which is vintage year minus one. Forecast errors are defined as forecast minus actual, so positive mean error means the forecast is optimistic relative to realized Brent.

Evaluation Metrics

The evaluation uses five metrics. Mean Error measures signed bias in dollars per barrel. Mean Percentage Error measures signed bias relative to realized price. Mean Absolute Error measures the typical absolute miss in dollars. Root Mean Squared Error penalizes large misses and is useful when extreme errors are especially damaging for project valuation. Mean Absolute Percentage Error measures typical percentage miss and helps compare accuracy across price regimes.

Using all five metrics matters because the stakeholder cares about two different properties: information and bias. A forecast can beat the benchmark on RMSE and still be too optimistic to use mechanically. That is exactly the main EIA finding. RMSE shows that EIA contains useful information beyond the no-change rule. ME and MPE show that this informational value does not eliminate the need for a planning haircut or governance adjustment.

The report avoids relying only on directional accuracy because the planning decision is fundamentally a level problem. Upstream economics depend on expected price levels over project-relevant years, not merely whether prices rise or fall. The report also avoids highly complex statistical tests because the realized long-horizon sample is small. Instead, it uses transparent forecast-error measures that are easy to audit in the GitHub appendix.

Benchmark Results

The first result is that EIA beats the martingale benchmark on RMSE at both horizons. At the 3-year horizon, EIA RMSE is 34.97 compared with 42.99 for the martingale. At the 5-year horizon, EIA RMSE is 34.32 compared with 47.34. This indicates that EIA long-term forecasts contain information that is not captured by simply carrying forward the latest completed annual Brent price.

However, the same table shows substantial optimism. EIA mean error is +20.44 dollars per barrel at three years and +26.64 at five years. In percentage terms, this is +33.0 percent at three years and +36.5 percent at five years. For project valuation, that is economically large. A planning deck that systematically overshoots realized Brent by roughly one-third can materially change screening decisions and reported project attractiveness.

The martingale benchmark is also positively biased in this sample, but it is less accurate on RMSE. This pattern supports a balanced conclusion: the institutional forecast is better than a naive rule, but it still requires disciplined adjustment. A stakeholder should not interpret EIA's RMSE advantage as permission to adopt the published forecast literally.

model	horizon_years	n_forecasts	ME	MPE	MAE	RMSE	MAPE
EIA	3	11	20.439	0.330	26.006	34.974	0.388
EIA	5	9	26.641	0.365	28.087	34.322	0.381
Martingale	3	11	17.251	0.276	31.657	42.989	0.438
Martingale	5	9	13.795	0.248	38.902	47.342	0.512

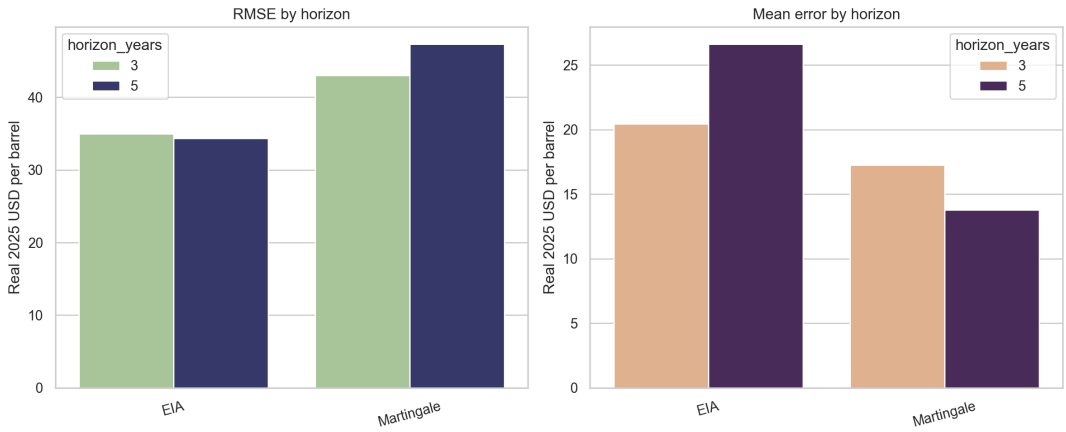

Figure 1. EIA and martingale benchmark performance.

Bias-Correction Results

The bias-correction exercise subtracts the historical mean EIA error within each horizon, estimated recursively using only prior realized errors. This design is important because it avoids look-ahead bias. For a given forecast origin, the correction uses only errors that would have been known at that time. The exercise therefore represents a feasible governance overlay rather than an ex post fitting exercise.

The correction improves RMSE modestly at three years and materially at five years. At three years, RMSE falls from 34.97 to 33.65. At five years, RMSE falls from 34.32 to 25.36 and MAPE falls to 23.5 percent. The correction is not perfect: it overcorrects the mean error into negative territory in the realized sample. Still, the performance gain demonstrates that historical optimism is not only descriptive; it is exploitable for practical planning.

For an upstream planning team, the implication is governance-oriented. The firm does not need to replace EIA with a complicated black-box model. A simple, documented, horizon-specific conservatism overlay can move the planning deck closer to realized performance. The exact haircut should be re-estimated as more vintages realize, but the principle is supported by the evidence.

model	horizon_years	n_forecasts	ME	MPE	MAE	RMSE	MAPE
Bias-Corrected EIA	3	10	-27.929	-0.315	29.255	33.650	0.341
Bias-Corrected EIA	5	8	-19.332	-0.212	20.533	25.359	0.235

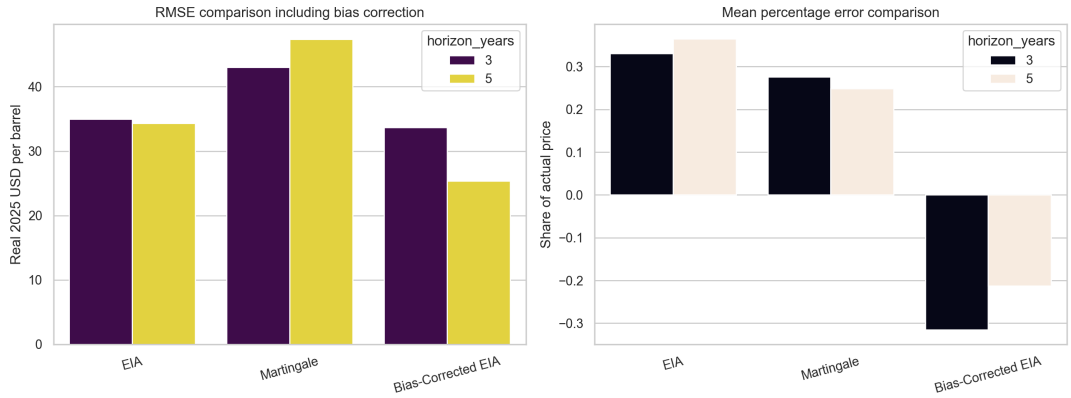

Figure 2. Bias-corrected EIA performance.

Rolling Model Results

The rolling econometric models test whether simple public-data models can improve on the institutional forecast. The Rolling AR model uses lagged annual real Brent prices. The Rolling AR plus Futures model adds release-month front-month Brent futures and a real basis proxy. Both models are estimated in expanding-window fashion so each forecast uses only prior information.

The models do not beat EIA in the current implementation. Rolling AR RMSE is 48.95 at the 3-year horizon and 51.24 at the 5-year horizon. Rolling AR plus Futures RMSE is 52.51 at three years and 53.23 at five years. These results do not mean market information is irrelevant. Rather, they show that a simple public front-month proxy is not enough to outperform the EIA long-horizon outlook in this sample.

This finding is useful because it prevents a common mistake: assuming that a reduced-form model is automatically better because it is data-driven. The EIA forecast may embed expert judgment, macro assumptions, supply-demand modeling, and policy expectations that are not captured by lagged annual Brent prices and front-month futures. The rolling models remain valuable as diagnostic baselines, but they should not replace the institutional forecast.

model	horizon_years	n_forecasts	ME	MPE	MAE	RMSE	MAPE
Rolling AR	3	26	10.132	0.157	35.328	48.952	0.447
Rolling AR	5	24	-5.407	-0.019	35.948	51.238	0.379
Rolling AR + Futures	3	9	17.960	0.325	40.110	52.511	0.549

model	horizon_years	n_forecasts	ME	MPE	MAE	RMSE	MAPE
Rolling AR + Futures	5	7	7.572	0.244	36.614	53.230	0.539

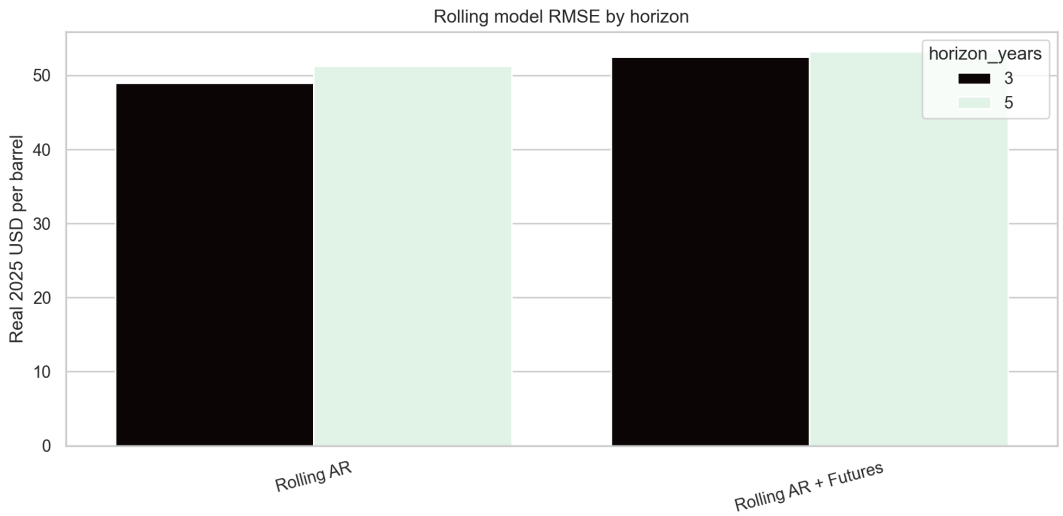

Figure 3. Rolling model RMSE comparison.

CPI Diagnostic

One review concern was whether CPI conversion could distort the real Brent analysis. The project addresses this by computing CPI-Brent relationship diagnostics. Nominal Brent growth and CPI inflation are positively related, with correlation around 0.519. Real Brent growth and CPI inflation also remain positively related, with correlation around 0.476. The average gap between real-2025 and nominal annual Brent is about 23.63 dollars per barrel.

These diagnostics suggest that the CPI adjustment is not mechanically creating the main bias result. The conversion standardizes units across vintages and target years, but the substantive conclusion remains about forecast performance relative to realized Brent. In the report, the CPI conversion is best interpreted as a comparability tool rather than a causal model of oil prices.

The project nevertheless treats CPI as a limitation. Real-dollar analysis can improve comparability, but it may not match every stakeholder's planning practice. Some companies evaluate nominal cash flows with explicit inflation assumptions. Others use real price decks for screening. The report's real 2025 dollar approach is transparent and consistent, but users should adapt the unit convention to their valuation framework.

Metric	Value	Interpretation
corr_nominal_brent_level_vs_cpi_level	0.753	Level correlation is descriptive only and may reflect common time trends.
corr_nominal_brent_growth_vs_cpi_inflation	0.519	Year-over-year co-movement between nominal Brent growth and CPI inflation.
corr_real_brent_growth_vs_cpi_inflation	0.476	Checks whether CPI comovement remains after converting Brent into real terms.
avg_deflation_gap_usd_per_bbl	23.632	Average difference between real-2025 and nominal annual Brent prices.
median_deflation_gap_usd_per_bbl	21.267	Median real-versus-nominal adjustment across annual observations.

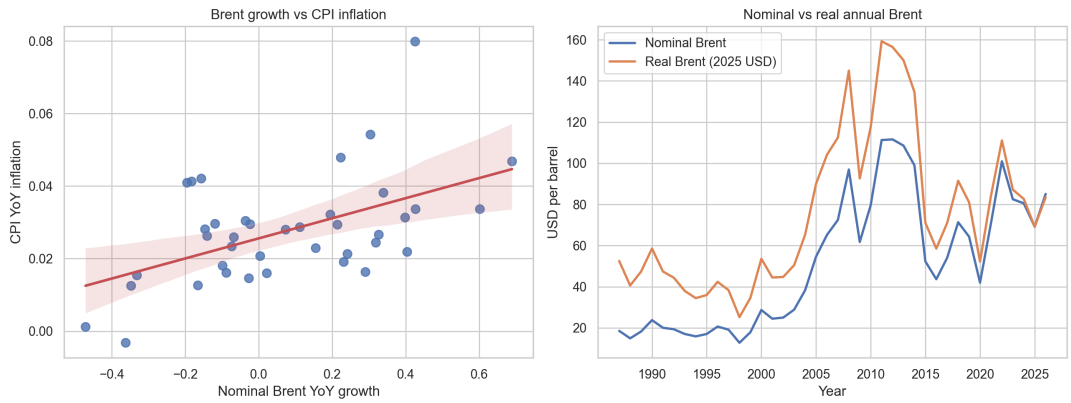

Figure 4. CPI and Brent relationship diagnostics.

Limitations

The first limitation is sample size. Long-horizon forecast evaluation is constrained by the need for realized target years. The 3-year and 5-year sample has only 20 observations across realized horizons. This limits formal inference and makes the estimates sensitive to large oil price shocks. The report therefore emphasizes transparent decision-oriented metrics instead of overstated statistical precision.

The second limitation is data availability. EIA provides a clean public archive, but other institutional forecasts are harder to integrate. IEA historical long-horizon Brent vintages were investigated but not included because the needed vintage-level series were not available in a clean public machine-readable panel within the project window. World Bank data are included as supplementary material, but the forecast object is crude oil average rather than exact Brent.

The third limitation is the futures extension. The project uses a public Brent front-month futures proxy rather than a full historical long-dated Brent futures strip. Long-horizon project economics would ideally use maturity-specific forward curve information. The current futures proxy is useful as a proof of concept but should not be interpreted as a definitive test of market-curve forecasting power.

The fourth limitation is that project-level cash-flow data are not included. The report evaluates forecasts, not project rankings. It can infer that optimistic forecasts may inflate NPV and IRR, but it cannot directly measure whether bias changes the ranking of specific projects under capital constraints. That would require a project-level panel with production profiles, cost assumptions, fiscal terms, and investment timing.

Recommendations

First, stakeholders should continue using EIA as an informational baseline. The forecast beats the martingale benchmark on RMSE, which means it contains useful signal beyond a no-change rule. Discarding EIA would ignore a public institutional forecast that performs better than a simple carry-forward benchmark.

Second, stakeholders should not use EIA mechanically. The realized sample shows persistent optimistic bias. A planning team should treat the published EIA forecast as a starting point rather than the final internal price deck. The adjustment should be explicit, documented, and reviewed as new realized errors become available.

Third, the best current governance improvement is a horizon-specific bias overlay. A simple recursive historical correction improved 5-year performance substantially. The exact correction should be refined with additional vintages and stakeholder risk tolerance, but the principle is clear: forecast governance should account for historical optimism.

Fourth, rolling AR-style models should remain supplementary diagnostics. They can provide a transparent comparison and help challenge the planning deck, but they should not replace EIA in this version. Future work should test longer-dated futures curves and project-level valuation impacts before recommending a model-based replacement.

Appendix A: Live GitHub Repository and File Map

The newly uploaded GitHub repository is the live appendix for this report. It contains the complete project folder, including source code, raw data, processed data, figures, final report materials, and presentation materials.

Repository item	Live link	Purpose
Repository root	https://github.com/jingrong129/INDENG-290	Complete uploaded project folder
Source code	https://github.com/jingrong129/INDENG-290/tree/main/src/oil_forecast_project	Pipeline package and analysis modules
Scripts	https://github.com/jingrong129/INDENG-290/tree/main/scripts	Pipeline entry point and report/presentation generation scripts
Processed data	https://github.com/jingrong129/INDENG-290/tree/main/data/processed	CSV tables used in the report
Raw EIA data	https://github.com/jingrong129/INDENG-290/tree/main/data/raw/eia	AEO Table 12 workbook archive
Outputs	https://github.com/jingrong129/INDENG-290/tree/main/outputs	Figures, workbook, PDF report, and PPTX deck
Report markdown	https://github.com/jingrong129/INDENG-290/blob/main/reports/final_project_report.md	Original markdown report source

Appendix B: Data Dictionary and Output Tables

This appendix documents the main processed files used in the report. The files are intentionally stored as CSV outputs so that the appendix can be audited without running the Python code.

File	Description
eia_brent_forecasts.csv	Full EIA vintage-target panel after unit conversion and benchmark merge.
actual_brent_annual.csv	Annual realized Brent averages from FRED, including nominal and real 2025 price fields.
annual_cpi.csv	Annual CPI averages and conversion factors into real 2025 dollars.
release_month_features.csv	Release-month spot and front-month futures features.
benchmark_metrics.csv	EIA versus martingale performance by 3-year and 5-year horizons.
bias_corrected_metrics.csv	Recursive bias-corrected EIA performance metrics.
rolling_model_metrics.csv	Rolling AR and Rolling AR plus Futures performance metrics.
cpi_brent_summary.csv	Diagnostic correlations and deflation-gap summaries.
analysis_outputs.xlsx	Consolidated workbook containing all primary tables.

Key fields include vintage_year, release_date, target_year, horizon_years, forecast_real_2025_usd_per_bbl, actual_real_2025_usd_per_bbl, random_walk_forecast_real_2025, forecast_error, and random_walk_error. Forecast

error is forecast minus actual; positive values indicate optimism relative to realized Brent.

Benchmark Metrics CSV

model	horizon_years	n_forecasts	ME	MPE	MAE	RMSE	MAPE
EIA	3.000	11.000	20.439	0.330	26.006	34.974	0.388
EIA	5.000	9.000	26.641	0.365	28.087	34.322	0.381
Martingale	3.000	11.000	17.251	0.276	31.657	42.989	0.438
Martingale	5.000	9.000	13.795	0.248	38.902	47.342	0.512

Bias-Corrected Metrics CSV

model	horizon_years	n_forecasts	ME	MPE	MAE	RMSE	MAPE
Bias-Corrected EIA	3.000	10.000	-27.929	-0.315	29.255	33.650	0.341
Bias-Corrected EIA	5.000	8.000	-19.332	-0.212	20.533	25.359	0.235

Rolling Model Metrics CSV

model	horizon_years	n_forecasts	ME	MPE	MAE	RMSE	MAPE
Rolling AR	3.000	26.000	10.132	0.157	35.328	48.952	0.447
Rolling AR	5.000	24.000	-5.407	-0.019	35.948	51.238	0.379
Rolling AR + Futures	3.000	9.000	17.960	0.325	40.110	52.511	0.549
Rolling AR + Futures	5.000	7.000	7.572	0.244	36.614	53.230	0.539

Appendix C: Reproducibility Protocol

The repository is organized so that the full analysis can be rerun from the command line. The pipeline downloads or reads raw public data, writes processed CSV files, exports an Excel workbook, and regenerates figures and summaries.

Clone or download the repository from https://github.com/jingrong129/INDENG-290.

Install the package dependencies with python3 -m pip install -e . from the repository root.

Run python3 scripts/run_pipeline.py to rebuild processed data, metrics, figures, and the analysis workbook.

Inspect data/processed for CSV tables and outputs/ for figures, workbook, report, and presentation materials.

Run pytest to execute the unit tests covering CPI conversion, annual Brent aggregation, release-month feature construction, evaluation-panel filtering, benchmark metrics, and bias correction.

Reproducibility is especially important because the project evaluates forecast vintages. The code preserves release dates, avoids using future realized errors in the bias correction, and uses expanding-window model estimation for rolling forecasts.

Pipeline entry point: scripts/run_pipeline.py

```
from pathlib import Path
import sys


PROJECT_ROOT = Path(__file__).resolve().parents[1]
SRC_DIR = PROJECT_ROOT / "src"
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))

from oil_forecast_project.pipeline import run_pipeline


if __name__ == "__main__":
    run_pipeline()
```

Appendix D: Core Code Excerpts

The following excerpts are included to make the appendix self-contained. The full source files are available in the GitHub repository; these excerpts show the parts of the pipeline most directly connected to the report results.

EIA vintage parser: src/oil_forecast_project/data_sources/eia.py

```
from __future__ import annotations

import re
from dataclasses import dataclass
from io import BytesIO
from urllib.parse import urljoin

import pandas as pd
import requests
from bs4 import BeautifulSoup
from openpyxl import load_workbook
```

```
from oil_forecast_project.config import RAW_DIR


CURRENT_AEO_URL = "https://www.eia.gov/outlooks/aeo/tables_ref.php"
ARCHIVE_AEO_URL = "https://www.eia.gov/outlooks/archive/aeo{yy}/tables_ref.php"
EIA_HEADERS = {"User-Agent": "Mozilla/5.0"}
EIA_RELEASE_DATES = {
    2013: "2013-04-15",
    2014: "2014-05-07",
    2015: "2015-04-14",
    2016: "2016-09-15",
    2017: "2017-01-05",

    2018: "2018-02-06",
    2019: "2019-01-24",
    2020: "2020-01-29",
    2021: "2021-02-03",
    2022: "2022-03-03",
    2023: "2023-03-16",
    2025: "2025-04-15",
    2026: "2026-04-08",
}


@dataclass(frozen=True)

class EiaVintageSpec:
    vintage_year: int
    page_url: str


def _page_url_for_vintage(vintage_year: int) -> str:
    if vintage_year == 2026:
        return CURRENT_AEO_URL
    return ARCHIVE_AEO_URL.format(yy=str(vintage_year)[-2:])


def list_eia_vintages(start_year: int = 2013, end_year: int = 2026) -> list[EiaVintageSpec]:
    return [
        EiaVintageSpec(vintage_year=year, page_url=_page_url_for_vintage(year))
        for year in range(start_year, end_year + 1)
        if year in EIA_RELEASE_DATES
    ]


def _get(url: str) -> requests.Response:
    response = requests.get(url, headers=EIA_HEADERS, timeout=60)
    response.raise_for_status()
    return response

def _extract_release_date(html: str) -> pd.Timestamp:
    candidates = []
    for pattern in [
        r"Release Dates?:.*?([A-Za-z]+\s+\d{1,2})(?:\s*-\s*[A-Za-z]+\s+\d{1,2})?,\s*(\d{4})",
        r"Release Dates?:.*?([A-Za-z]+\s+\d{1,2},\s+\d{4})",
        r"Release date:.*?([A-Za-z]+\s+\d{1,2},\s+\d{4})",
    ]:
        match = re.search(pattern, html, flags=re.IGNORECASE | re.DOTALL)
        if not match:
            continue
```

```
        if len(match.groups()) == 2:

candidates.append(f"{match.group(1)}, {match.group(2)}")
        else:
            candidates.append(match.group(1))
    if not candidates:
        soup = BeautifulSoup(html, "html.parser")
        for container in soup.select(".release-dates"):
            date_spans = container.select("span.date")
            for date_span in date_spans:
                text = " ".join(date_span.get_text(" ", strip=True).split())
                if re.match(r"[A-Za-z]+\s+\d{1,2},\s+\d{4}", text):
                    candidates.append(text)
                    break
            if candidates:
                break
    if not candidates:
        raise ValueError("Unable to parse EIA release date.")
    return pd.to_datetime(candidates[0])


def _extract_release_date_for_page(page_url: str, html: str) -> pd.Timestamp:
    try:
        return _extract_release_date(html)
    except ValueError:
        base_url = page_url.rsplit("/", 1)[0] + "/"

        parent_html = _get(base_url).text
        return _extract_release_date(parent_html)


def _default_table12_xlsx_url(vintage_year: int) -> str:
    if vintage_year == 2026:
        return "https://www.eia.gov/outlooks/aeo/excel/aeotab12.xlsx"
    base = f"https://www.eia.gov/outlooks/archive/aeo{str(vintage_year)[-2:]}/excel/"
    filename = "aeotab12.xlsx" if vintage_year >= 2022 else "aeotab_12.xlsx"
    return urljoin(base, filename)

def _extract_table12_xlsx_url(vintage_year: int, page_url: str, html: str) -> str:
    anchor_pattern = re.compile(
        r"Table 12\.\s*Petroleum(?: and Other Liquids)? Prices.*?XLSX",
        re.IGNORECASE | re.DOTALL,
    )
    if not anchor_pattern.search(html):
        return _default_table12_xlsx_url(vintage_year)
    anchors = re.findall(r'<a href="([^"]+)">([^<]*)</a>', html, flags=re.IGNORECASE)
    for idx, (_, text) in enumerate(anchors):
        text_clean = " ".join(text.split())
        if text_clean.startswith("Table 12."):
            for href, next_text in anchors[idx + 1 : idx + 4]:
                if next_text.strip().upper() in {"XLSX", "XLS"}:
                    return urljoin(page_url, href)
    return _default_table12_xlsx_url(vintage_year)


def _detect_year_row(sheet: pd.DataFrame) -> int:
    for idx in range(min(20, len(sheet))):
        values = [value for value in sheet.iloc[idx].tolist() if pd.notna(value)]
        year_hits = 0
        for value in values:
```

```
            if isinstance(value, (int, float)) and 1900 <= int(value) <= 2100:
                year_hits += 1

            elif hasattr(value, "year"):
                year_hits += 1
        if year_hits >= 6:
            return idx
    raise ValueError("Unable to find year header row in EIA table.")


def _extract_dollar_year(sheet: pd.DataFrame) -> int:
    for idx in range(min(20, len(sheet))):
        row_text = " ".join(str(value) for value in sheet.iloc[idx].tolist() if pd.notna(value))
        match = re.search(r"\((\d{4}) dollars per barrel\)", row_text, flags=re.IGNORECASE)
        if match:

return int(match.group(1))
    raise ValueError("Unable to parse EIA dollar year.")


def _extract_brent_row(sheet: pd.DataFrame) -> pd.Series:
    mask = sheet.astype(str).apply(lambda col: col.str.contains("Brent Spot", case=False, na=False)).any(ax
    if not mask.any():
        raise ValueError("Unable to locate Brent row in EIA table.")
    return sheet.loc[mask].iloc[0]


def _normalize_year(value: object) -> int:
    if hasattr(value, "year"):
        return int(value.year)
    return int(value)


def fetch_eia_brent_vintages(start_year: int = 2013, end_year: int = 2026) -> pd.DataFrame:
    records: list[dict[str, object]] = []
    for spec in list_eia_vintages(start_year=start_year, end_year=end_year):
        release_date = pd.Timestamp(EIA_RELEASE_DATES[spec.vintage_year])
        xlsx_url = _default_table12_xlsx_url(spec.vintage_year)

        raw_path = RAW_DIR / "eia" / f"aeo_{spec.vintage_year}_table12.xlsx"

        raw_path.parent.mkdir(parents=True, exist_ok=True)
        if raw_path.exists():
```

Dataset construction: src/oil_forecast_project/datasets.py

```
from __future__ import annotations

import numpy as np
import pandas as pd

from oil_forecast_project.analysis.metrics import summarize_errors
from oil_forecast_project.config import PROCESSED_DIR
from oil_forecast_project.data_sources.eia import fetch_eia_brent_vintages
from oil_forecast_project.data_sources.fred import fetch_brent_spot_daily, fetch_cpi_monthly
from oil_forecast_project.data_sources.market import fetch_brent_front_month_futures
from oil_forecast_project.data_sources.world_bank import fetch_world_bank_oil_average_forecasts
from oil_forecast_project.io import write_csv

TARGET_REAL_YEAR = 2025
```

```
def build_annual_cpi(cpi_monthly: pd.DataFrame) -> pd.DataFrame:
    annual = (
        cpi_monthly.assign(year=cpi_monthly["date"].dt.year)
        .groupby("year", as_index=False)["value"]
        .mean()
        .rename(columns={"value": "cpi_avg"})
    )

    target_cpi = annual.loc[annual["year"] == TARGET_REAL_YEAR, "cpi_avg"].iloc[0]
    annual["to_2025_factor"] = target_cpi / annual["cpi_avg"]
    return annual


def build_monthly_cpi(cpi_monthly: pd.DataFrame) -> pd.DataFrame:
    target_cpi = cpi_monthly.loc[cpi_monthly["date"].dt.year == TARGET_REAL_YEAR, "value"].mean()
    monthly = cpi_monthly.copy()
    monthly["year"] = monthly["date"].dt.year
    monthly["month"] = monthly["date"].dt.month
    monthly["to_2025_factor"] = target_cpi / monthly["value"]
    return monthly[["date", "year", "month", "value", "to_2025_factor"]]

def build_actual_brent_annual(brent_daily: pd.DataFrame, annual_cpi: pd.DataFrame) -> pd.DataFrame:
    actual = (
        brent_daily.dropna(subset=["value"])
        .assign(year=brent_daily["date"].dt.year)
        .groupby("year", as_index=False)["value"]
        .mean()
        .rename(columns={"value": "actual_nominal_usd_per_bbl"})
        .merge(annual_cpi[["year", "to_2025_factor"]], on="year", how="left")
    )
    actual["actual_real_2025_usd_per_bbl"] = actual["actual_nominal_usd_per_bbl"] * actual["to_2025_factor"
    return actual


def build_release_month_features(
    brent_daily: pd.DataFrame,
    futures_daily: pd.DataFrame,
    monthly_cpi: pd.DataFrame,
    eia_vintages: pd.DataFrame,
) -> pd.DataFrame:
    known_releases = eia_vintages[["vintage_year", "release_date"]].drop_duplicates().copy()
    min_year = int(min(brent_daily["date"].dt.year.min(), futures_daily["date"].dt.year.min()))
    max_year = int(max(brent_daily["date"].dt.year.max(), futures_daily["date"].dt.year.max()))
    release_rows = []
    known_release_map = dict(zip(known_releases["vintage_year"], known_releases["release_date"]))

    for year in range(min_year, max_year + 1):
        if year in known_release_map:
            release_date = pd.Timestamp(known_release_map[year])
        else:
            release_date = pd.Timestamp(f"{year}-01-15")
        release_rows.append({"vintage_year": year, "release_date": release_date})
    release_months = pd.DataFrame(release_rows)
    release_months["year"] = release_months["release_date"].dt.year
    release_months["month"] = release_months["release_date"].dt.month

spot_monthly = (
        brent_daily.dropna(subset=["value"])
        .assign(year=brent_daily["date"].dt.year, month=brent_daily["date"].dt.month)
        .groupby(["year", "month"], as_index=False)["value"]
```

```
        .mean()
        .rename(columns={"value": "spot_release_month_nominal"})
    )
    futures_monthly = (
        futures_daily.dropna(subset=["close"])
        .assign(year=futures_daily["date"].dt.year, month=futures_daily["date"].dt.month)
        .groupby(["year", "month"], as_index=False)["close"]
        .mean()
        .rename(columns={"close": "futures_release_month_nominal"})
    )

    features = (
        release_months.merge(spot_monthly, on=["year", "month"], how="left")
        .merge(futures_monthly, on=["year", "month"], how="left")
        .merge(monthly_cpi[["year", "month", "to_2025_factor"]], on=["year", "month"], how="left")
    )
    features["spot_release_month_real_2025"] = features["spot_release_month_nominal"] * features["to_2025_fa
    features["futures_release_month_real_2025"] = features["futures_release_month_nominal"] * features["to_
    return features[
        [

            "vintage_year",
            "release_date",
            "spot_release_month_nominal",
            "futures_release_month_nominal",
            "spot_release_month_real_2025",
            "futures_release_month_real_2025",
        ]
    ]


def build_eia_forecast_dataset() -> dict[str, pd.DataFrame]:
    eia = fetch_eia_brent_vintages(start_year=2013, end_year=2026)

brent_daily = fetch_brent_spot_daily()
    cpi_monthly = fetch_cpi_monthly()
    futures_daily = fetch_brent_front_month_futures()
    world_bank = fetch_world_bank_oil_average_forecasts()

    annual_cpi = build_annual_cpi(cpi_monthly)
    monthly_cpi = build_monthly_cpi(cpi_monthly)
    actual = build_actual_brent_annual(brent_daily, annual_cpi)
    release_features = build_release_month_features(brent_daily, futures_daily, monthly_cpi, eia)

    eia = eia.merge(
        annual_cpi.rename(columns={"year": "table12_base_year"})[["table12_base_year", "to_2025_factor"]],
        on="table12_base_year",
        how="left",
    )
    eia["forecast_real_2025_usd_per_bbl"] = eia["forecast_real_base"] * eia["to_2025_factor"]
    eia["horizon_years"] = eia["target_year"] - eia["vintage_year"]
    eia["benchmark_anchor_year"] = eia["vintage_year"] - 1
    eia = eia.merge(actual[["year", "actual_real_2025_usd_per_bbl"]].rename(columns={"year": "target_year"}
    eia = eia.merge(
        actual[["year", "actual_real_2025_usd_per_bbl"]].rename(
            columns={"year": "benchmark_anchor_year", "actual_real_2025_usd_per_bbl": "random_walk_forecast_
        ),
        on="benchmark_anchor_year",

        how="left",
```

```
    )
    eia = eia.merge(release_features, on=["vintage_year", "release_date"], how="left")
    eia["forecast_error"] = eia["forecast_real_2025_usd_per_bbl"] - eia["actual_real_2025_usd_per_bbl"]
    eia["random_walk_error"] = eia["random_walk_forecast_real_2025"] - eia["actual_real_2025_usd_per_bbl"]

    for df_name, df in {
        "eia_brent_forecasts.csv": eia,
        "actual_brent_annual.csv": actual,
        "annual_cpi.csv": annual_cpi,
        "release_month_features.csv": release_features,
        "world_bank_oil_average_forecasts.csv": world_bank,

}.items():
        write_csv(df, PROCESSED_DIR / df_name)

    return {
        "eia": eia,
        "actual": actual,
        "annual_cpi": annual_cpi,
        "monthly_cpi": monthly_cpi,
        "release_features": release_features,
        "world_bank": world_bank,
        "futures_daily": futures_daily,
        "brent_daily": brent_daily,
    }


def build_evaluation_panel(eia: pd.DataFrame) -> pd.DataFrame:
    panel = eia.loc[eia["horizon_years"].isin([3, 5])].dropna(
        subset=["actual_real_2025_usd_per_bbl", "forecast_real_2025_usd_per_bbl", "random_walk_forecast_rea
    ).copy()
    panel["absolute_error"] = panel["forecast_error"].abs()
    panel["squared_error"] = panel["forecast_error"] ** 2
    panel["benchmark_absolute_error"] = panel["random_walk_error"].abs()
    panel["benchmark_squared_error"] = panel["random_walk_error"] ** 2
    return panel.sort_values(["horizon_years", "vintage_year"]).reset_index(drop=True)

def write_benchmark_summary(panel: pd.DataFrame) -> pd.DataFrame:
    eia_summary = summarize_errors(
        panel,
        provider_label="EIA",
        forecast_col="forecast_real_2025_usd_per_bbl",
        error_col="forecast_error",
    )
    random_walk_frame = panel[
        ["horizon_years", "actual_real_2025_usd_per_bbl", "random_walk_forecast_real_2025", "random_walk_er
    ].rename(

columns={
            "random_walk_forecast_real_2025": "forecast_real_2025_usd_per_bbl",
            "random_walk_error": "forecast_error",
        }
    )
    rw_summary = summarize_errors(
        random_walk_frame,
        provider_label="Martingale",
        forecast_col="forecast_real_2025_usd_per_bbl",
        error_col="forecast_error",
```

Forecast metrics: src/oil_forecast_project/analysis/metrics.py

```
from __future__ import annotations

import math

import pandas as pd


def _rmse(series: pd.Series) -> float:
    return math.sqrt((series**2).mean())


def summarize_errors(
    frame: pd.DataFrame,
    provider_label: str,
    forecast_col: str,
    error_col: str,
) -> pd.DataFrame:
    records: list[dict[str, float | int | str]] = []
    for horizon, group in frame.groupby("horizon_years"):
        error = group[error_col]
        actual = group["actual_real_2025_usd_per_bbl"]
        forecast = group[forecast_col]
        percentage_error = error / actual
        mape = ((forecast - actual).abs() / actual).mean()

        records.append(
            {
                "model": provider_label,
                "horizon_years": int(horizon),
                "n_forecasts": int(len(group)),
                "ME": error.mean(),
                "MPE": percentage_error.mean(),
                "MAE": error.abs().mean(),
                "RMSE": _rmse(error),
                "MAPE": mape,
            }
        )
```

return pd.DataFrame.from_records(records)

Bias correction and CPI diagnostic: src/oil_forecast_project/analysis/additional.py

```
from __future__ import annotations

import pandas as pd

from oil_forecast_project.analysis.metrics import summarize_errors


def build_bias_corrected_eia_forecasts(panel: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    corrected = panel.sort_values(["horizon_years", "release_date", "vintage_year"]).copy()
    grouped_errors = corrected.groupby("horizon_years")["forecast_error"]
    corrected["historical_mean_error"] = grouped_errors.transform(lambda series: series.shift(1).expanding(
    corrected["historical_error_count"] = grouped_errors.transform(lambda series: series.shift(1).expanding
    corrected["bias_corrected_forecast_real_2025_usd_per_bbl"] = (
        corrected["forecast_real_2025_usd_per_bbl"] - corrected["historical_mean_error"]
    )
    corrected["bias_corrected_error"] = (
```

```
        corrected["bias_corrected_forecast_real_2025_usd_per_bbl"] - corrected["actual_real_2025_usd_per_bb
    )

    evaluated = corrected.dropna(subset=["bias_corrected_forecast_real_2025_usd_per_bbl"]).copy()
    metrics = summarize_errors(
        evaluated,
        provider_label="Bias-Corrected EIA",
        forecast_col="bias_corrected_forecast_real_2025_usd_per_bbl",

        error_col="bias_corrected_error",
    )
    return corrected, metrics


def summarize_cpi_brent_relationship(actual: pd.DataFrame, annual_cpi: pd.DataFrame) -> tuple[pd.DataFrame,
    joined = actual.merge(annual_cpi[["year", "cpi_avg"]], on="year", how="inner").sort_values("year").copy
    joined["cpi_inflation_yoy"] = joined["cpi_avg"].pct_change()
    joined["nominal_brent_growth_yoy"] = joined["actual_nominal_usd_per_bbl"].pct_change()
    joined["real_brent_growth_yoy"] = joined["actual_real_2025_usd_per_bbl"].pct_change()
    joined["deflation_gap_usd_per_bbl"] = joined["actual_real_2025_usd_per_bbl"] - joined["actual_nominal_u

growth = joined.dropna(
        subset=["cpi_inflation_yoy", "nominal_brent_growth_yoy", "real_brent_growth_yoy"]
    ).copy()
    summary = pd.DataFrame(
        [
            {
                "metric": "corr_nominal_brent_level_vs_cpi_level",
                "value": joined["actual_nominal_usd_per_bbl"].corr(joined["cpi_avg"]),
                "interpretation": "Level correlation is descriptive only and may reflect common time trends
            },
            {
                "metric": "corr_nominal_brent_growth_vs_cpi_inflation",
                "value": growth["nominal_brent_growth_yoy"].corr(growth["cpi_inflation_yoy"]),
                "interpretation": "Year-over-year co-movement between nominal Brent growth and CPI inflatio
            },
            {
                "metric": "corr_real_brent_growth_vs_cpi_inflation",
                "value": growth["real_brent_growth_yoy"].corr(growth["cpi_inflation_yoy"]),
                "interpretation": "Checks whether CPI comovement remains after converting Brent into real t
            },
            {
                "metric": "avg_deflation_gap_usd_per_bbl",
                "value": joined["deflation_gap_usd_per_bbl"].mean(),
                "interpretation": "Average difference between real-2025 and nominal annual Brent prices.",

            },
            {
                "metric": "median_deflation_gap_usd_per_bbl",
                "value": joined["deflation_gap_usd_per_bbl"].median(),
                "interpretation": "Median real-versus-nominal adjustment across annual observations.",
            },
        ]
    )
    return joined, summary
```

Rolling model implementation: src/oil_forecast_project/analysis/models.py

```
from __future__ import annotations
```

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

from oil_forecast_project.analysis.metrics import summarize_errors
from oil_forecast_project.config import PROCESSED_DIR
from oil_forecast_project.io import write_csv


def _build_origin_feature_table(actual: pd.DataFrame, release_features: pd.DataFrame) -> pd.DataFrame:
    prices = actual[["year", "actual_real_2025_usd_per_bbl"]].rename(columns={"year": "price_year"})
    release = release_features.copy()
    release["origin_year"] = release["vintage_year"]
    release["last_complete_year"] = release["origin_year"] - 1

    feature_rows: list[dict[str, float | int | pd.Timestamp]] = []
    price_map = dict(zip(prices["price_year"], prices["actual_real_2025_usd_per_bbl"]))
    release_map = release.set_index("origin_year").to_dict("index")
    min_year = int(prices["price_year"].min())
    max_year = int(prices["price_year"].max())
    for origin_year in range(min_year + 6, max_year + 1):
        if origin_year not in release_map:

            continue
        last_complete = origin_year - 1
        needed_years = [last_complete - i for i in range(5)]
        if not all(year in price_map for year in needed_years):
            continue
        release_row = release_map[origin_year]
        lag1 = price_map[last_complete]
        lag2 = price_map[last_complete - 1]
        lag3 = price_map[last_complete - 2]
        lag5_mean = np.mean([price_map[last_complete - i] for i in range(5)])
        mom_1y = lag1 / lag2 - 1
        futures_real = release_row.get("futures_release_month_real_2025")

basis = futures_real - lag1 if pd.notna(futures_real) else np.nan
        feature_rows.append(
            {
                "origin_year": origin_year,
                "release_date": release_row["release_date"],
                "lag1": lag1,
                "lag2": lag2,
                "lag3": lag3,
                "lag5_mean": lag5_mean,
                "mom_1y": mom_1y,
                "futures_release_month_real_2025": futures_real,
                "basis_real_2025": basis,
            }
        )
    return pd.DataFrame(feature_rows)


def _rolling_predictions(
    origin_features: pd.DataFrame,
    actual: pd.DataFrame,
    horizon: int,
    feature_cols: list[str],
    model_name: str,
) -> pd.DataFrame:
```

```
    target_map = dict(zip(actual["year"], actual["actual_real_2025_usd_per_bbl"]))
    rows: list[dict[str, float | int | str | pd.Timestamp]] = []
    model_frame = origin_features.copy()
    model_frame["target_year"] = model_frame["origin_year"] + horizon
    model_frame["actual_real_2025_usd_per_bbl"] = model_frame["target_year"].map(target_map)
    model_frame = model_frame.dropna(subset=["actual_real_2025_usd_per_bbl"] + feature_cols).sort_values("o

    for _, row in model_frame.iterrows():
        train = model_frame[model_frame["origin_year"] < row["origin_year"]].dropna(subset=feature_cols)
        if len(train) < 5:
            continue
        reg = LinearRegression()

reg.fit(train[feature_cols], train["actual_real_2025_usd_per_bbl"])
        forecast = float(reg.predict(pd.DataFrame([row[feature_cols]], columns=feature_cols))[0])
        error = forecast - row["actual_real_2025_usd_per_bbl"]
        rows.append(
            {
                "model": model_name,
                "origin_year": int(row["origin_year"]),
                "release_date": row["release_date"],
                "horizon_years": horizon,
                "target_year": int(row["target_year"]),
                "forecast_real_2025_usd_per_bbl": forecast,
                "actual_real_2025_usd_per_bbl": float(row["actual_real_2025_usd_per_bbl"]),
                "forecast_error": error,
            }
        )
    return pd.DataFrame(rows)


def fit_rolling_models(actual: pd.DataFrame, release_features: pd.DataFrame) -> tuple[pd.DataFrame, pd.Data
    origin_features = _build_origin_feature_table(actual, release_features)
    prediction_frames = []
    specs = [
        ("Rolling AR", ["lag1", "lag2", "lag3"]),
        ("Rolling AR + Futures", ["lag1", "lag2", "lag3", "futures_release_month_real_2025", "basis_real_20

    ]
    for horizon in [3, 5]:
        for model_name, feature_cols in specs:
            prediction_frames.append(_rolling_predictions(origin_features, actual, horizon, feature_cols, m

    predictions = pd.concat([frame for frame in prediction_frames if not frame.empty], ignore_index=True)
    metrics = []
    for model_name, group in predictions.groupby("model"):
        metrics.append(
            summarize_errors(
                group.assign(horizon_years=group["horizon_years"]),
                provider_label=model_name,

forecast_col="forecast_real_2025_usd_per_bbl",
                error_col="forecast_error",
            )
        )
    metrics_df = pd.concat(metrics, ignore_index=True) if metrics else pd.DataFrame()
    write_csv(predictions, PROCESSED_DIR / "rolling_model_predictions.csv")
    write_csv(metrics_df, PROCESSED_DIR / "rolling_model_metrics.csv")
    return predictions, metrics_df
```

Appendix E: Validation Tests

The project includes unit tests for the most decision-relevant transformations. These tests reduce the risk that the report results are artifacts of incorrect CPI conversion, annual aggregation, benchmark construction, or look-ahead bias in the recursive adjustment.

Dataset tests: tests/test_datasets.py

```
import pandas as pd
import pytest

from oil_forecast_project import datasets


def test_build_annual_cpi_converts_to_2025_dollars():
    cpi_monthly = pd.DataFrame(
        {
            "date": pd.to_datetime(["2024-01-01", "2024-02-01", "2025-01-01", "2025-02-01"]),
            "value": [100.0, 100.0, 200.0, 200.0],
        }
    )

    annual = datasets.build_annual_cpi(cpi_monthly)

    factors = dict(zip(annual["year"], annual["to_2025_factor"]))
    assert factors[2024] == pytest.approx(2.0)
    assert factors[2025] == pytest.approx(1.0)


def test_build_actual_brent_annual_aggregates_and_deflates():
    brent_daily = pd.DataFrame(
        {

            "date": pd.to_datetime(["2024-01-01", "2024-01-02", "2024-01-03", "2025-01-01"]),
            "value": [70.0, 90.0, None, 100.0],
        }
    )
    annual_cpi = pd.DataFrame({"year": [2024, 2025], "to_2025_factor": [1.5, 1.0]})

    actual = datasets.build_actual_brent_annual(brent_daily, annual_cpi)

    row_2024 = actual.loc[actual["year"] == 2024].iloc[0]
    assert row_2024["actual_nominal_usd_per_bbl"] == pytest.approx(80.0)
    assert row_2024["actual_real_2025_usd_per_bbl"] == pytest.approx(120.0)

def test_build_release_month_features_uses_known_release_month_and_real_prices():
    brent_daily = pd.DataFrame(
        {
            "date": pd.to_datetime(["2024-04-01", "2024-04-15", "2025-01-15"]),
            "value": [80.0, 100.0, 120.0],
        }
    )
    futures_daily = pd.DataFrame(
        {
            "date": pd.to_datetime(["2024-04-03", "2024-04-17", "2025-01-15"]),
            "close": [90.0, 110.0, 130.0],
        }
```

```
    )
    monthly_cpi = pd.DataFrame(
        {
            "year": [2024, 2025],
            "month": [4, 1],
            "to_2025_factor": [1.25, 1.0],
        }
    )
    eia_vintages = pd.DataFrame({"vintage_year": [2024], "release_date": pd.to_datetime(["2024-04-20"])})

    features = datasets.build_release_month_features(brent_daily, futures_daily, monthly_cpi, eia_vintages)

    row_2024 = features.loc[features["vintage_year"] == 2024].iloc[0]
    assert row_2024["spot_release_month_nominal"] == pytest.approx(90.0)
    assert row_2024["futures_release_month_nominal"] == pytest.approx(100.0)
    assert row_2024["spot_release_month_real_2025"] == pytest.approx(112.5)
    assert row_2024["futures_release_month_real_2025"] == pytest.approx(125.0)


def test_build_evaluation_panel_filters_horizons_and_requires_realized_values():
    eia = pd.DataFrame(
        {
            "vintage_year": [2020, 2021, 2022, 2023],

"target_year": [2023, 2026, 2026, 2025],
            "horizon_years": [3, 5, 4, 2],
            "forecast_real_2025_usd_per_bbl": [80.0, 90.0, 95.0, 70.0],
            "actual_real_2025_usd_per_bbl": [100.0, 120.0, 130.0, None],
            "random_walk_forecast_real_2025": [75.0, 85.0, 88.0, 60.0],
            "forecast_error": [-20.0, -30.0, -35.0, None],
            "random_walk_error": [-25.0, -35.0, -42.0, None],
        }
    )

    panel = datasets.build_evaluation_panel(eia)

    assert list(panel["horizon_years"]) == [3, 5]
    assert list(panel["vintage_year"]) == [2020, 2021]
    assert panel.loc[0, "absolute_error"] == pytest.approx(20.0)
    assert panel.loc[1, "squared_error"] == pytest.approx(900.0)
    assert panel.loc[0, "benchmark_absolute_error"] == pytest.approx(25.0)
    assert panel.loc[1, "benchmark_squared_error"] == pytest.approx(1225.0)


def test_write_benchmark_summary_compares_eia_and_random_walk(tmp_path, monkeypatch):
    monkeypatch.setattr(datasets, "PROCESSED_DIR", tmp_path)
    panel = pd.DataFrame(
        {

            "horizon_years": [3, 3],
            "actual_real_2025_usd_per_bbl": [100.0, 200.0],
            "forecast_real_2025_usd_per_bbl": [110.0, 180.0],
            "forecast_error": [10.0, -20.0],
            "random_walk_forecast_real_2025": [90.0, 190.0],
            "random_walk_error": [-10.0, -10.0],
        }
    )

    summary = datasets.write_benchmark_summary(panel)
```

```
    assert (tmp_path / "benchmark_metrics.csv").exists()

assert set(summary["model"]) == {"EIA", "Martingale"}
    eia = summary.loc[summary["model"] == "EIA"].iloc[0]
    random_walk = summary.loc[summary["model"] == "Martingale"].iloc[0]
    assert eia["ME"] == pytest.approx(-5.0)
    assert eia["MAE"] == pytest.approx(15.0)
    assert random_walk["ME"] == pytest.approx(-10.0)
    assert random_walk["RMSE"] == pytest.approx(10.0)
```

Additional-analysis tests: tests/test_additional.py

```
import pandas as pd
import pytest

from oil_forecast_project.analysis.additional import build_bias_corrected_eia_forecasts, summarize_cpi_bren


def test_build_bias_corrected_eia_forecasts_uses_only_prior_errors():
    panel = pd.DataFrame(
        {
            "vintage_year": [2018, 2019, 2020, 2018, 2019, 2020],
            "release_date": pd.to_datetime(
                ["2018-01-01", "2019-01-01", "2020-01-01", "2018-01-01", "2019-01-01", "2020-01-01"]
            ),
            "horizon_years": [3, 3, 3, 5, 5, 5],
            "forecast_real_2025_usd_per_bbl": [100.0, 110.0, 120.0, 90.0, 95.0, 105.0],
            "actual_real_2025_usd_per_bbl": [90.0, 100.0, 115.0, 80.0, 90.0, 100.0],
            "forecast_error": [10.0, 10.0, 5.0, 10.0, 5.0, 5.0],
        }
    )

    corrected, metrics = build_bias_corrected_eia_forecasts(panel)

    horizon_3_2020 = corrected.loc[(corrected["horizon_years"] == 3) & (corrected["vintage_year"] == 2020)]
    assert horizon_3_2020["historical_mean_error"] == pytest.approx(10.0)

    assert horizon_3_2020["bias_corrected_forecast_real_2025_usd_per_bbl"] == pytest.approx(110.0)
    assert horizon_3_2020["bias_corrected_error"] == pytest.approx(-5.0)
    assert set(metrics["horizon_years"]) == {3, 5}


def test_summarize_cpi_brent_relationship_returns_expected_metrics():
    actual = pd.DataFrame(
        {
            "year": [2023, 2024, 2025],
            "actual_nominal_usd_per_bbl": [80.0, 100.0, 110.0],
            "actual_real_2025_usd_per_bbl": [90.0, 105.0, 110.0],
        }

)
    annual_cpi = pd.DataFrame({"year": [2023, 2024, 2025], "cpi_avg": [280.0, 300.0, 330.0]})

    relationship, summary = summarize_cpi_brent_relationship(actual, annual_cpi)

    assert "cpi_inflation_yoy" in relationship.columns
    assert "nominal_brent_growth_yoy" in relationship.columns
    assert set(summary["metric"]) == {
        "corr_nominal_brent_level_vs_cpi_level",
        "corr_nominal_brent_growth_vs_cpi_inflation",
```

```
        "corr_real_brent_growth_vs_cpi_inflation",
        "avg_deflation_gap_usd_per_bbl",
        "median_deflation_gap_usd_per_bbl",
    }
```

Appendix F: Decision Translation for Upstream Planning

The statistical finding can be translated into a price-deck governance workflow. A planning team should first collect institutional outlooks, including EIA, and record release dates and units. It should then convert the outlook into the internal valuation unit, estimate recent realized forecast errors by horizon, and apply a documented haircut or bias correction. The corrected deck should be reviewed alongside downside cases, especially for projects with high upfront capital intensity and long payback periods.

The governance point is that bias adjustment should not be hidden inside ad hoc analyst judgment. If the firm uses a modified EIA deck, the modification should be explicit: what historical window was used, what horizon was corrected, whether the correction was dollar-based or percentage-based, and whether the same correction applies across all project types. This creates an audit trail and avoids inconsistent treatment across competing projects.

The report's results also support scenario design. EIA can serve as the central external baseline, while the bias-corrected series can serve as a conservative planning case. The martingale benchmark can serve as a simple reference case. Stress cases can be constructed below these levels depending on company risk tolerance, financing constraints, and portfolio exposure. The important lesson is that forecast evaluation should inform scenario architecture rather than produce a single mechanically accepted number.

Finally, the project highlights the importance of continuing data collection. Each additional realized target year can update the historical error distribution. The bias correction should therefore be treated as a living governance tool rather than a one-time report output. The GitHub repository structure makes this update process straightforward because new vintages and realized prices can be added to the same pipeline.

Appendix G: Metric Interpretation Notes

Mean Error is the best direct measure of optimistic or pessimistic planning bias. A positive value means the forecast sits above realized Brent on average. Mean Percentage Error translates that same signed miss into economic scale relative to

realized prices.

RMSE is especially relevant for capital allocation because large forecast misses can dominate project economics. A few large misses can matter more than many small misses if they occur around investment sanction decisions or debt-service stress periods.

MAPE is intuitive for nontechnical stakeholders, but it should be interpreted carefully when realized prices are unusually low. In this sample it is still useful because Brent prices remain economically meaningful and far from zero.

Appendix H: Suggested Future Extensions

A future version should collect a historical long-dated Brent futures strip, not only front-month futures. Long-dated contracts would better match 3-year and 5-year planning horizons.

A future version should integrate project-level economics. With production curves, capex, opex, fiscal terms, and price sensitivities, the analysis could test whether forecast bias changes project ranking and capital allocation decisions.

A future version should test additional institutional outlooks if clean vintage data become available. This would clarify whether optimism is specific to EIA or common across public long-term energy forecasts.